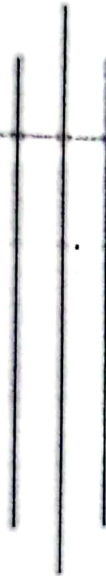




MADAN BHANDARI MEMORIAL COLLEGE

New Baneshor, Kathmandu

School of Science and Technology



Lab report of Design and Analysis of Algorithms

Submitted By:

Aswin phuyal

BSc.CSIT(5th sem)

Section: B

Submitted To:

lokesh gupta

INDEX

Date	LAB	Signature	Remark

Question 1:

Write algorithms to compute Factorial using both recursive and iterative methods and analyze their complexity task.

(a) Write an iterative algorithm for Factorial of n

(b) Write a recursive algorithm for Factorial of n

(c) Perform time and space complexity analysis under RAM model.

(d) Compare iterative vs recursive approach

Introduction:

Factorial of a number is the product of all positive integers less than or equal to that number. It is widely used in mathematical probability and algorithm design. It can be calculated using both iterative and recursive methods.

Objectives

- To implement Factorial using iterative & recursive method
- To analyze their time & space complexity
- To compare both approaches

Theory:

Factorial of a number n is defined as:

$$n! = n \times (n-1) \times (n-2) \times \dots \times 1$$

Base case

$$0! = 1$$

$$1! = 1$$

There are two approaches

- 1) Iterative: Uses loops
- 2) Recursive: Function calls itself

Algorithms

(2) Iterative Algorithm:

(1) Start

(2) Read n

(3) Set Fact = 1

(4) For i = 1 to n

Fact = Fact * i

(5) Print Fact

(6) Stop

U.L.R

(B) Recursive Factorial Algorithm

- (1) Start
- (2) If $n=0$ or $n=1$ then return 1
- (3) Else return $n * \text{factorial}(n-1)$
- (4) Stop

Source code:

```
#include <stdio.h>
int factorial-recu(int n)
{
    if (n==0 || n==1)
        return 1;
    else
        return n * factorial-recu(n-1);
}

int main()
{
    int n, i;
    long long fact = 1;
    printf("Enter a number: ");
    scanf("%d", &n);
    for (i=2; i <= n; i++)
    {
        fact *= i;
    }

    printf("Iterative Factorial: %lld\n", fact);
    printf("Recursive Factorial: %d\n", factorial-recu(n));
    return 0;
}
```

Complexity Analysis (RAM model)

method	Time complexity	Space complexity
Iterative	$O(n)$	$O(1)$
Recursive	$O(n)$	$O(n)$

Output

Enter a number = 6
Iterative Factorial: 720
Recursive Factorial: 720

Conclusion

- Both methods give same result
- Iterative method approach is more memory efficient
- Recursive is easier to understand but uses more space

Reference

introduce to Algorithms
DAA textbooks

Question - 2

Understand and apply Big-O, Big- Ω and Big- Θ notation with graphical interpretation. Consider the following function

$$f(n) = 3n^2 + 5n + 10$$

Plot geometrical interpretation for Big-O, Big- Ω , Big- Θ curves

Write a C program with nested loops & determine its asymptotic complexity

Introduction

Asymptotic notations are used to describe algorithm efficiency as input size increases

Objective:

- Understand Big-O, Big- Ω , Big- Θ
- Graphically interpret the function
- Analyze a nested loop program

Theory

Given:

$$f(n) = 3n^2 + 5n + 10$$

As n becomes larger
Dominant term $= n^2$

So

$$\text{Big-O} = O(n^2)$$

$$\text{Big-}\Omega = \Omega(n^2)$$

$$\text{Big-}\Theta = \Theta(n^2)$$

Graphical interpretation

~~But~~

First we identify the dominant term in $f(n)$
 $= 3n^2 + 5n + 10$ since n^2 grows faster than n or a constant, the complexity is quadratic. We use $g(n) = n^2$ as our reference function for all three notation.

For Big-O notation, we seek a constant c_2 and no such that $f(n) \leq c_2 \cdot g(n)$ we can pick $c_2 = 8$ because for $n \geq 4$

$$3n^2 + 5n + 10 \leq 3n^2 + 5n^2 + 10n^2 = 18n^2$$

However a tighter $c_2 = 8$ works effectively for the plot as $3n^2 + 5n + 20 \leq 8n^2$ simplifies to $5n^2 - 5n - 20 \geq 0$ which holds for $n \geq 2$

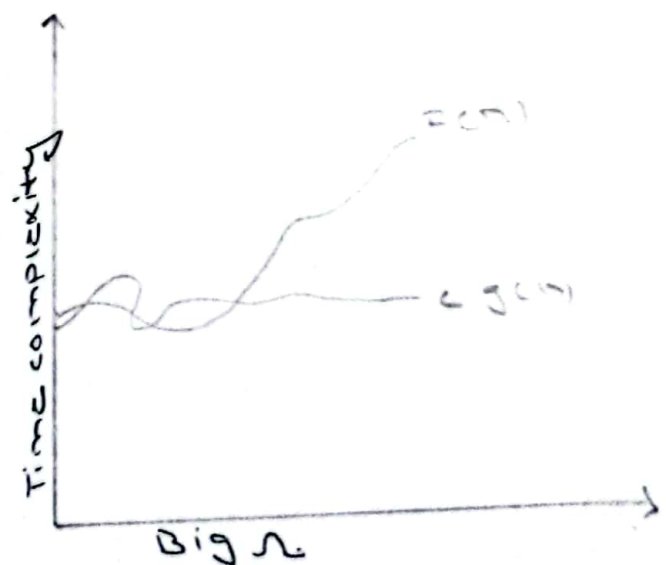
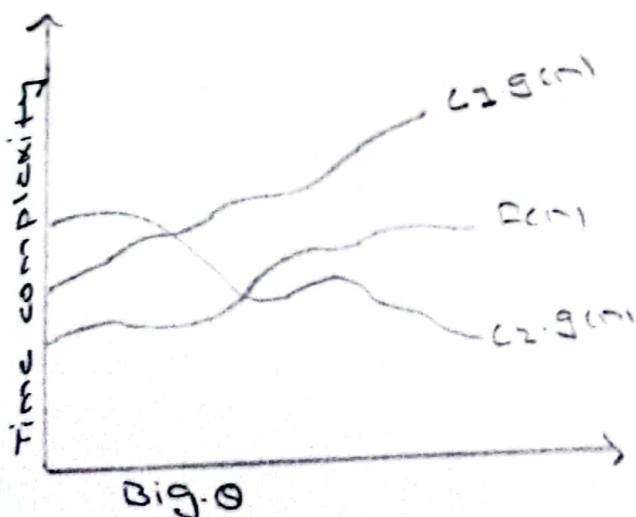
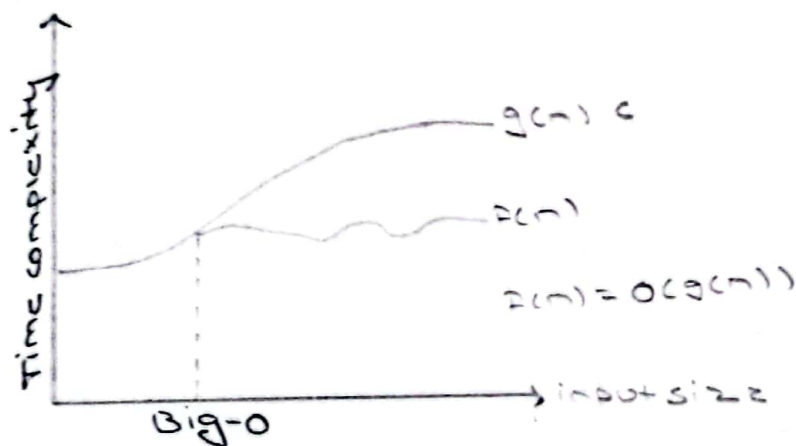
For Big-omega notation, we seek a constant c_1 and n_0 such that $f(n) \geq c_1 \cdot g(n)$. Since $3n^2$ is always less than $3n^2 + 5n + 20$ for $n > 0$ we can safely choose $c_1 = 2$

$$2n^2 \leq 3n^2 + 5n + 20$$

This inequality holds for all $n \geq 1$

Geometrical representation

- Big-O: The curve $8n^2$ stays above $f(n)$
- Big Ω : The curve $2n^2$ stays below $f(n)$
- Big Θ : $f(n)$ resides between $2n^2$ and $8n^2$ for all n beyond the critical point n_0



Source code

```
#include <stdio.h>
```

```
int main()
```

```
{
```

```
int n, i, j;
```

```
printf("Enter n: ");
```

```
scanf("%d", &n);
```

```
for(i=1; i<=n; i++)
```

```
{ for(j=1; j<=n; j++)
```

```
{ printf("Hello");
```

```
}
```

```
printf("\n");
```

```
}
```

```
return 0;
```

```
}
```

Complexity

Outer Loop: n

Inner Loop: n

Total operations: $n \times n = n^2$

Time complexity = $O(n^2)$

Space complexity = $O(1)$

Output

```
for n=2
```

```
Hello Hello
```

```
Hello Hello
```

Conclusion:

- Big-O gives worst case
- Big- Ω gives best case
- Big Θ gives exact growth rate
- Nested loops usually produce quadratic complexity

Reference:

Algorithm Design - Aho, Hopcroft & Ullman

QUESTION 3

Solve recurrences from recursive algorithms using different methods give recurrence

$$T(n) = 2T(n/2) + n$$

Implement recursive merge sort and verify recurrence complexity

Introduction

Recursive algorithms are analyzed using recurrence relations

Objective:

- Solve recurrence using different methods
- Implement merge sort
- Verify complexity

Theory

Given:

$$T(n) = 2T(n/2) + n$$

Using master theorem:

$$a=2, b=2, f(n)=n$$

$$n(\log_b a) = n(\log_2 2)$$

$$= n^1$$

$$= n$$

Case 2:

$$T(n) = \Theta(n \log n)$$

Algorithm: merge sort

1) If array size = 1 → return

2) Divide array into two halves

3) Recursively sort both halves

4) Merge the sorted halves

Source code:

```
#include <stdio.h>
```

```
void merge(int arr[], int l, int m, int r)
```

```
{
```

```
    int i, j, k;
```

```
    int n1 = m - l + 1;
```

```
    int n2 = r - m;
```

```
    int L[n1], R[n2];
```

```

for(i=0; i<n/2; i++)
{
    L[i] = arr[L+1+i];
}
for(j=0; j<n/2; j++)
{
    R[j] = arr[m+1+j];
    i=j=0;
    k=L;
}
while(i<n/2 && j<n/2)
{
    if(L[i] <= R[j])
        arr[k++] = L[i++];
    else
        arr[k++] = R[j++];
}
while(i<n/2)
    arr[k++] = L[i++];
while(j<n/2)
    arr[k++] = R[j++];
}

void mergesort(int arr[], int l, int r)
{
    if(l < r)
    {
        int m = (l+r)/2;
        mergesort(arr, l, m);
        mergesort(arr, m+1, r);
        merge(arr, l, m, r);
    }
}

int main()
{
    int arr[] = {3, 2, 5, 6, 4};
    int n = 5;
    mergesort(arr, 0, n-1);
    printf("sorted array: \n");
    for(i=0; i<n; i++)
    {
        printf("%d", arr[i]);
    }
    return 0;
}

```

Conclusion:

Efficient for large datasets

Recurrence solve using Master Theorem

Merge sort has time complexity $O(n \log n)$

Reference

CLRS Algorithm Books

Question: 4

Implement basic iterative algorithms and analyze complexity

Write an iterative C program for

Write an iterative program to compute

Analyze time and space complexity of both Algorithms

Introduction:

Iterative algorithms use loops instead of recursion

Objective

- Implementation GCD and Fibonacci iteratively
- Analyze complexity

Theory

Euclidean Algorithm

Repeatedly replace:

$a = b$

$b = a \% b$

until $b = 0$

Fibonacci

$F(n) = F(n-1) + F(n-2)$

Algorithm

GCD

1. Read a, b

2. While $b \neq 0$

temp = b

$b = a \% b$

$a = \text{temp}$

3. Print a

Fibonacci

1. Read n

2. Set $a = 0, b = 1$

3. Repeat n times:

$c = a + b$

$a = b$

$b = c$

4. Print sequence

Source code:

```
#include <stdio.h>
int main()
{
    int a, b, temp, n, i, x=0, y=1, z;
    printf("Enter two numbers");
    scanf("%d %d", &a, &b);
    while (b != 0)
    {
        temp = b;
        b = a % b;
        a = temp;
    }
    printf("GCD: %d\n", a);
    printf("Enter n: ");
    scanf("%d", &n);
    printf("Fibonacci:");
    for (i=0; i < n; i++)
    {
        printf("%d ", x);
        z = x + y;
        x = y;
        y = z;
    }
    return 0;
}
```

complexity

Algorithm	Time	Space
GCD	$O(\log n)$	$O(1)$ $O(1)$
Fibonacci	$O(n)$	$O(1)$

Output

Enter two numbers: 12 18

GCD = 6

Enter n: 5

Fibonacci: 0 1 1 2 3

Conclusion

- Iterative GCD is very efficient
- Iterative Fibonacci avoids recursion overhead

Reference

• Data Structures by Weiss

Question 5: Implement sequential search and three sorting algorithms with complexity comparison implement in C and analyze

- Best case
- Worst case
- Average case

Implement the following sorting algorithms

- Bubble sort
- Selection sort
- Insertion sort

For each sort

- Count number of comparison & swaps
- Analyze time complexity

Compare performance for:

- Sorted array
- Reverse array
- Random array

Introduction

Searching and sorting are fundamental operations in algorithms

Objective

- Implement linear search
- Implement Bubble, Selection and Insertion sort
- Compare their complexities

Theory

Linear search: search element one by one
time complexity

- Best: $O(1)$
- Worst: $O(n)$
- Average: $O(n)$

Algorithms

Linear search

1. Read array and key
2. For each element
3. If match $\rightarrow$ print position

Bubble sort

1. compare adjacent element
2. swap if out of order
3. Repeat until sorted

Selection sort

1. Find element which is smallest
2. swap with first position
3. Repeat

Insertion sort

1. Take element
2. insert into sorted part

Source code:

```
#include <stdio.h>
void bubbleSort(int arr[], int n)
{
    int i, j, temp;
    for (i = 0; i < n - 1; i++)
    {
        for (j = 0; j < n - i - 1; j++)
        {
            temp = arr[j];
            arr[j] = arr[j + 1];
            arr[j + 1] = temp;
        }
    }
}
```

```
void selectionSort(int arr[], int n)
{
    int i, j, min, temp;
    for (i = 0; i < n - 1; i++)
    {
        min = i;
        for (j = i + 1; j < n; j++)
        {
            if (arr[j] < arr[min])
                min = j;
        }
        temp = arr[i];
        arr[i] = arr[min];
        arr[min] = temp;
    }
}
```

```
void insertionSort(int arr[], int n) {
```

```
    int i, key, j;
```

```
    for(i=1; i<n; i++)
```

```
    { key = arr[i];
```

```
      j = i-1;
```

```
      while(j >= 0 && arr[j] > key)
```

```
      { arr[j+1] = arr[j];
```

```
        j--;
```

```
      }
```

```
      arr[j+1] = key;
```

```
    } }
```

```
int main()
```

```
{ int arr[] = { 3, 4, 2, 1, 5 };
```

```
  int n = 5;
```

```
  bubbleSort(arr, n);
```

```
  printf("Sorted array: \n");
```

```
  for(i=0; i<n; i++)
```

```
  printf("%d", arr[i]);
```

```
  return 0;
```

```
}
```

complexity comparison

Algorithm	Best	Average	Worst
Linear search	$O(1)$	$O(n)$	$O(n)$
Bubble sort	$O(n)$	$O(n^2)$	$O(n^2)$
Selection sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion sort	$O(n)$	$O(n^2)$	$O(n^2)$

Output

Sorted array

1 2 3 4 5

Conclusion: Linear search is simple but slow for large data. Bubble & selection sort are inefficient. Insertion sort performs better on nearly sorted data.

Reference: Introduction to Algorithms
Data Structure textbook